

DSE — Distribuição Linux Mínima

Telemetria via MQTT com Yocto Project

Documentação Técnica de Projeto e Testes

Repositório: github.com/RaffaellaSantos/yocto_mqtt

Imagem Docker: `mqtt-yocto:v5`

Equipe:

Adriana Raffaella dos Santos Fonseca

Davi Aguiar Moreira

Henrique da Silva Furtado

1. Introdução

Este documento descreve o projeto de uma distribuição Linux mínima para sistemas embarcados, construída com o Yocto Project. O sistema executa um aplicativo Python que coleta métricas de telemetria (CPU, memória e disco) e as publica via protocolo MQTT para monitoramento remoto em tempo real.

A distribuição foi gerada a partir do repositório público da equipe e demonstrada em ambiente de emulação via Docker, tornando a validação independente de hardware dedicado.

1.1 Objetivos

- Construir distribuição Linux mínima com Yocto Project (OpenEmbedded Core)
- Incluir somente pacotes essenciais para execução da telemetria
- Desenvolver app Python com paho-mqtt e psutil para coleta e envio de métricas
- Garantir inicialização automática do app no boot via init script
- Demonstrar funcionamento em emulador Docker (imagem OCI mqtt-yocto:v5)

1.2 Tecnologias Utilizadas

Tecnologia	Versão	Função no Projeto
Yocto Project / OE-Core	Scarthgap / master	Geração da distribuição Linux
Python	3.14.4	Linguagem do app de telemetria
paho-mqtt	2.1.0	Cliente MQTT — publicação das métricas
psutil	7.2.2	Coleta de CPU, memória e disco
BusyBox	1.37.0	Shell e utilitários do sistema
OpenSSL	3.5.6	TLS para conexão segura com broker
Docker (OCI)	mqtt-yocto:v5	Emulação do sistema embarcado
MQTT Broker	broker.hivemq.com	Servidor de mensagens na nuvem

2. Repositório e Estrutura do Projeto

O código-fonte do projeto está disponível em:

https://github.com/RaffaellaSantos/yocto_mqtt

2.1 Estrutura de Diretórios

A árvore completa do repositório (2 níveis):

```
.
├── build/
│   ├── bitbake-cookerdaemon.log
│   ├── bitbake.lock
│   ├── cache/
│   ├── conf/
│   ├── init-build-env
│   ├── README
│   └── tmp/
├── config/
│   ├── config-upstream.json
│   └── sources-fixed-revisions.json
├── layers/
│   ├── bitbake/
│   ├── logs/
│   ├── meta-openembedded/
│   ├── meta-yocto/
│   ├── oe-init-build-env-dir → openembedded-core
│   ├── oe-scripts → openembedded-core/scripts
│   ├── openembedded-core/
│   ├── setup-build → openembedded-core/scripts/oe-setup-build
│   ├── sources-fixed-revisions.json → ../config/
│   └── yocto-docs/
├── meta-custom/ ← LAYER CUSTOMIZADA DA EQUIPE
│   ├── conf/
│   ├── COPYING.MIT
│   ├── README
│   ├── recipes-core/
│   └── recipes-example/
```

2.2 Layer Customizada — meta-custom

A layer meta-custom contém toda a lógica do projeto da equipe. Dentro dela, os arquivos mais importantes são:

```
meta-custom/
├── conf/
├── COPYING.MIT
├── README
├── recipes-core/
│   ├── images/
│   │   └── minimal-image.bb ← Define os pacotes da imagem
│   └── init/
│       ├── files/
│       │   ├── init-mqtt.py ← App Python de telemetria
│       │   ├── init-script.sh ← Script de inicialização
│       │   └── init-script.bb ← Recipe BitBake do app
```

A layer meta-custom é o coração do projeto. As outras layers (openembedded-core, meta-yocto) são dependências padrão do Yocto e não foram modificadas.

3. Processo de Criação da Distribuição

3.1 Metodologia

O processo de construção da distribuição seguiu a documentação oficial do Yocto Project. As etapas foram:

1. Instalação das dependências do host (Ubuntu/Debian)
2. Clone do repositório: `git clone https://git.openembedded.org/bitbake`
3. Configuração do ambiente de build via `oe-init-build-env`
4. Criação da layer customizada meta-custom com as recipes do projeto
5. Definição da imagem mínima em `minimal-image.bb`
6. Escrita do app Python (`init-mqtt.py`) e script de init (`init-script.sh`)
7. Build completo: `bitbake minimal-image`
8. Empacotamento da imagem como container OCI para Docker

3.2 Recipe da Imagem — `minimal-image.bb`

A imagem mínima é definida em `recipes-core/images/minimal-image.bb`. Ela herda de `core-image` e adiciona apenas os pacotes necessários para a telemetria:

```
require recipes-core/images/core-image-minimal.bb

IMAGE_INSTALL:append = " \
    busybox \
    python3-core \
    python3-json \
    python3-paho-mqtt \
    python3-psutil \
    init-script \
"
```

3.3 Recipe do App — `init-script.bb`

A recipe `init-script.bb` instrui o BitBake a instalar o app Python e o script de init no sistema de arquivos da imagem:

```
SUMMARY = "Init - Recipe"
LICENSE = "MIT"

LIC_FILES_CHKSUM =
"file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = " \
    file://init-script.sh \
    file://init-mqtt.py \
"
S = "${UNPACKDIR}"

do_install() {
    install -d ${D}/init
    install -m 0755 ${S}/init-script.sh ${D}/init/init-script.sh
    install -m 0755 ${S}/init-mqtt.py ${D}/init/init-mqtt.py
}

FILES:${PN} += "/init /init/init-script.sh /init/init-mqtt.py"
```

```
RDEPENDS:${PN} = "busybox"
```

4. Aplicativo de Telemetria — init-mqtt.py

4.1 Função do Aplicativo

O arquivo init-mqtt.py é o núcleo do projeto. Ele é responsável por coletar as métricas do sistema operacional usando a biblioteca psutil e publicá-las a cada 5 segundos no broker MQTT no formato JSON.

4.2 Código Fonte

```
#!/usr/bin/env python3
import time
import json
import socket
import psutil
import paho.mqtt.client as mqtt

# — Configurações —
BROKER_HOST = "broker.hivemq.com"
BROKER_PORT = 1883
TOPIC_BASE = "embarcados/telemetria/v3"
INTERVAL = 5
CLIENT_ID = "yocto-device-01"
#

def coletar_dados():
    origem_dispositivo = socket.gethostname()
    return {
        "user": origem_dispositivo,
        "cpu_percent": psutil.cpu_percent(interval=1),
        "mem_total_mb": round(psutil.virtual_memory().total / 1024**2, 2),
        "mem_used_mb": round(psutil.virtual_memory().used / 1024**2, 2),
        "mem_percent": psutil.virtual_memory().percent,
        "disk_total_mb": round(psutil.disk_usage("/").total / 1024**2, 2),
        "disk_free_mb": round(psutil.disk_usage("/").free / 1024**2, 2),
        "disk_percent": psutil.disk_usage("/").percent,
        "timestamp": int(time.time()),
    }

def on_connect(client, userdata, flags, rc, properties):
    if rc == 0:
        print("[MQTT] Conectado ao broker!")
    else:
        print(f"[MQTT] Falha na conexão, código: {rc}")

def main():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id=CLIENT_ID)
    client.on_connect = on_connect

    while True:
        try:
            client.connect(BROKER_HOST, BROKER_PORT, keepalive=60)
            client.loop_start()
            break
        except Exception as e:
            print(f"[MQTT] Erro ao conectar: {e}. Tentando em 5s...")
            time.sleep(5)

    print("[Telemetria] Iniciando coleta...")
    while True:
        try:
            dados = coletar_dados()
```

```

        payload = json.dumps(dados)
        client.publish(TOPIC_BASE, payload, qos=1)
        print(f"[Telemetria] Publicado: {payload}")
    except Exception as e:
        print(f"[Telemetria] Erro: {e}")
    time.sleep(INTERVAL)

if __name__ == "__main__":
    main()

```

4.3 Payload JSON Publicado

Exemplo de mensagem publicada no tópico MQTT a cada 5 segundos:

```

{
  "user": "192.10.2.1"
  "cpu_percent": 8.3,
  "mem_total_mb": 512.0,
  "mem_used_mb": 127.4,
  "mem_percent": 24.9,
  "disk_total_mb": 2048.0,
  "disk_free_mb": 1644.3,
  "disk_percent": 19.7,
  "timestamp": 1748021345
}

```

4.4 Script de Inicialização — init-script.sh

O init-script.sh é executado pelo sistema no boot. Ele verifica dependências, exibe informações do sistema e inicia o app Python em background:

```

#!/bin/sh

echo "\n ===== Minimal Image DSE ====="

mkdir -p /proc /sys /dev
mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev

echo "Configurando DNS..."
echo "nameserver 8.8.8.8" > /etc/resolv.conf

echo "Verificando dependencias do sistema..."
python3 --version
python3 -c "import paho.mqtt; import psutil; print('\n Ambiente MQTT e Telemetria: OK!)"

echo "Listando diretórios:"
ls /

echo "Iniciando client MQTT de telemetria..."
exec python3 -u /init/init-mqtt.py

```

⚠ As mensagens 'mount: permission denied (are you root?)' são normais ao rodar no Docker, pois o container não possui privilégios de kernel. O MQTT não é afetado.

5. Manifest de Pacotes da Imagem

A tabela abaixo lista todos os 47 pacotes presentes na imagem final `mqtt-yocto:v1`, conforme o manifest gerado pelo BitBake. Os pacotes marcados com ★ são os diretamente relacionados à telemetria MQTT.

#	Pacote	Arch	Versão	Função
1	busybox	x86-64-v3	1.37.0-r0	Shell, utilitários básicos do sistema
2	busybox-udhcp	x86-64-v3	1.37.0-r0	Cliente DHCP para obter IP via rede
3	ca-certificates	all	20260223-r0	Certificados CA para TLS/HTTPS
4	init-script	x86-64-v3	1.0-r0	Script de inicialização do projeto
5	ldconfig	x86-64-v3	2.43+git0+ce1013a197-r1	Gerenciador de cache de bibliotecas dinâmicas
6	libbz2-1	x86-64-v3	1.0.8-r0	Compressão bzip2
7	libc6	x86-64-v3	2.43+git0+ce1013a197-r1	Biblioteca C padrão (glibc)
8	libcrypto3	x86-64-v3	3.5.6-r0	Primitivas criptográficas (OpenSSL)
9	libedit0	x86-64-v3	20251016-3.1-r0	Edição de linha de comando
10	libexpat1	x86-64-v3	2.7.5-r0	Parser XML
11	libffi8	x86-64-v3	3.5.2-r0	Interface de função estrangeira (ctypes)
12	libgcc1	x86-64-v3	15.2.0-r0	Suporte runtime do GCC
13	liblzma5	x86-64-v3	5.8.2-r0	Compressão LZMA/XZ
14	libncurses5	x86-64-v3	6.6-r0	Interface de terminal (ncurses)
15	libpython3.14-1.0	x86-64-v3	3.14.4-r0	Biblioteca compartilhada do Python 3.14
16	libssl3	x86-64-v3	3.5.6-r0	TLS/SSL para conexão MQTT segura
17	libtinfo5	x86-64-v3	6.6-r0	Suporte terminal (terminfo)
18	libuuid1	x86-64-v3	2.41.3-r0	Geração de UUIDs
19	libz1	x86-64-v3	1.3.2-r0	Compressão zlib
20	libzstd1	x86-64-v3	1.5.7-r0	Compressão Zstandard
21	ncurses-terminfo-base	x86-64-v3	6.6-r0	Base de dados de terminais

2 2	openssl	x86-64-v3	3.5.6-r0	Toolkit SSL/TLS
2 3	openssl-bin	x86-64-v3	3.5.6-r0	Binários OpenSSL
2 4	openssl-conf	x86-64-v3	3.5.6-r0	Configuração OpenSSL
2 5	python3-compression	x86-64-v3	3.14.4-r0	Módulos de compressão Python
2 6	python3-core	x86-64-v3	3.14.4-r0	Núcleo do interpretador Python 3
2 7	python3-crypt	x86-64-v3	3.14.4-r0	Módulo de criptografia Python
2 8	python3-ctypes	x86-64-v3	3.14.4-r0	Interface C para Python
2 9	python3-datetime	x86-64-v3	3.14.4-r0	Manipulação de datas e timestamps
3 0	python3-email	x86-64-v3	3.14.4-r0	Processamento de e-mail
3 1	python3-io	x86-64-v3	3.14.4-r0	Operações de entrada/saída
3 2	python3-json	x86-64-v3	3.14.4-r0	Serialização JSON do payload MQTT
3 3	python3-logging	x86-64-v3	3.14.4-r0	Sistema de logs
3 4	python3-math	x86-64-v3	3.14.4-r0	Funções matemáticas
3 5	python3-mime	x86-64-v3	3.14.4-r0	Tipos MIME
3 6	python3-netclient	x86-64-v3	3.14.4-r0	Cliente de rede Python
3 7	python3-netserver	x86-64-v3	3.14.4-r0	Servidor de rede Python
3 8	python3-paho-mqtt	x86-64-v3	2.1.0-r0	★ Cliente MQTT — publicação da telemetria
3 9	python3-pickle	x86-64-v3	3.14.4-r0	Serialização de objetos Python
4 0	python3-psutil	x86-64-v3	7.2.2-r0	★ Coleta CPU, memória e disco
4 1	python3-resource	x86-64-v3	3.14.4-r0	Informações de recursos do sistema
4 2	python3-shell	x86-64-v3	3.14.4-r0	Utilitários shell Python
4 3	python3-stringold	x86-64-v3	3.14.4-r0	Compatibilidade string legado

4 4	python3-threading	x86-64-v3	3.14.4-r0	Suporte a threads (loop MQTT)
4 5	python3-xml	x86-64-v3	3.14.4-r0	Processamento XML
4 6	update-alternatives -opkg	x86-64-v3	0.7.0-r0	Gerenciador de alternativas
4 7	util-linux-flock	x86-64-v3	2.41.3-r0	Bloqueio de arquivos (flock)

★ *python3-paho-mqtt (2.1.0) e python3-psutil (7.2.2) são as dependências centrais do app de telemetria. Todos os demais pacotes são dependências indiretas do Python ou do sistema mínimo.*

6. Demonstração em Emulador — Docker

6.1 Sobre a Imagem OCI

A imagem `mqtt-yocto:v1` é uma imagem OCI gerada pelo processo de build do Yocto e empacotada como arquivo `.tar` para distribuição. Ela representa exatamente o sistema de arquivos que rodaria em hardware embarcado real.

Atributo	Valor
Arquivo	<code>mqtt-yocto-v5.tar</code>
Tag Docker	<code>mqtt-yocto:v5</code>
Formato	OCI (Open Container Initiative)
Init script	<code>/init/init-script.sh</code>
Shell	BusyBox ash (prompt: <code>~ #</code>)
App Python	<code>/init/init-mqtt.py</code>
Arquitetura	<code>x86-64-v3</code>

6.2 Passo a Passo para Executar

Passo 1 — Extrair o arquivo (se compactado)

```
# Windows PowerShell
Expand-Archive mqtt-yocto-v5.zip -DestinationPath .

# Linux / macOS
tar -xf mqtt-yocto-v5.tar.gz
```

Passo 2 — Carregar a imagem no Docker

```
docker load -i mqtt-yocto-v5.tar
```

Saída esperada:

```
Loaded image: mqtt-yocto:v5
```

Passo 3 — Verificar que a imagem foi carregada

```
docker images | grep mqtt-yocto
```

Saída esperada:

```
mqtt-yocto    v5    <image_id>    ...    <size>
```

Passo 4 — Executar o container

```
docker run -it --rm mqtt-yocto:v5 /init/init-script.sh
```

Saída esperada na tela:

```
===== Minimal Image DSE =====
mount: permission denied (are you root?)    ← normal no Docker
mount: permission denied (are you root?)    ← ignorar
mount: permission denied (are you root?)    ← ignorar
Verificando dependencias do sistema...
```

```
Python 3.14.4
Ambiente MQTT e Telemetria: OK!
Listando diretorios:
bin  dev  etc  init  lib  proc  run  sbin  sys  usr  var
Iniciando client MQTT de telemetria...
Iniciando console interativo...
~ #
```

7. MQTT — Conectividade e Monitoramento

7.1 Protocolo MQTT

O MQTT (Message Queuing Telemetry Transport) é um protocolo leve de mensagens baseado em publish/subscribe, projetado para dispositivos com recursos limitados e redes instáveis — características centrais de sistemas embarcados. Utiliza TCP e consome banda mínima.

7.2 Brokers Disponíveis

Broker	Host	MQTT	WebSocket	Recomendado
HiveMQ ★	broker.hivemq.com	1883	8884	Sim — mais estável
Mosquitto	test.mosquitto.org	1883	8081	Backup
Local	localhost / 10.0.2.2	1883	9001	Ambiente isolado

7.3 Receber Dados no HiveMQ Web Client

9. Acessar: <https://www.hivemq.com/demos/websocket-client/>
10. Host: broker.hivemq.com | Port: 8884
11. Clicar em Connect — aguardar status verde
12. Em Subscriptions, digitar o tópico: embarcados/telemetria/v3
13. Clicar Subscribe — os dados JSON aparecem em tempo real

8. Procedimentos de Teste

8.1 Teste 1 — Carregamento da Imagem Docker

Item	Descrição
Objetivo	Verificar que a imagem OCI carrega corretamente no Docker
Comando	<code>docker load -i mqtt-yocto-v5.tar</code>
Esperado	Loaded image: mqtt-yocto:v5
Verificação	<code>docker images grep mqtt-yocto</code>
Status	Passou

8.2 Teste 2 — Inicialização do Sistema

Item	Descrição
Objetivo	Verificar boot do sistema e init script
Comando	<code>docker run -it --rm mqtt-yocto:v1 /init/init-script.sh</code>
Esperado	Mensagem 'Ambiente MQTT e Telemetria: OK!' e prompt ~ #
Observação	Erros de mount são normais no Docker — não afetam MQTT
Status	Passou

8.3 Teste 3 — Verificação de Dependências

Item	Descrição
Objetivo	Confirmar Python 3.14 e libs de telemetria instalados
Comando	<code>python3 --version / python3 -c 'import paho.mqtt.client, psutil'</code>
Esperado	Python 3.14.4 / sem ImportError
Status	Passou — confirmado pelo init-script.sh

8.4 Teste 4 — Publicação MQTT

Item	Descrição
Objetivo	Confirmar publicação de dados no broker
Comando	<code>python3 /init/init-mqtt.py</code>
Esperado	[MQTT] Conectado! / [Telemetria] Publicado: {...} a cada 5s
Status	Passou

8.5 Teste 5 — Recepção em Computador Externo

Item	Descrição
Objetivo	Confirmar que outro computador recebe os dados via MQTT
Ferramenta	HiveMQ Web Client — https://www.hivemq.com/demos/websocket-client/
Config	broker.hivemq.com : 8884 tópico: embarcados/telemetria/v3
Esperado	JSON com CPU, memória e disco atualizando a cada 5s
Status	Passou — validado com dois computadores distintos

8.6 Teste 6 — Reconexão Automática

Item	Descrição
Objetivo	Verificar retry automático ao falhar conexão com broker
Procedimento	Usar broker inválido, observar retry; corrigir e verificar reconexão
Esperado	Mensagens de retry a cada 5s; conexão restabelecida automaticamente
Status	Passou

9. Conclusão

O projeto atingiu todos os requisitos estabelecidos para a disciplina de Desenvolvimento de Sistemas Embarcados:

- Distribuição Linux mínima gerada com Yocto Project / OpenEmbedded Core
- Imagem com apenas 47 pacotes — nenhum utilitário desnecessário
- App Python 3.14 com paho-mqtt 2.1.0 e psutil 7.2.2 para coleta e envio de telemetria
- Inicialização automática via init-script.sh ao ligar o sistema
- Demonstração funcional em emulador Docker (mqtt-yocto:v5)
- Validação de recepção em computador externo via HiveMQ Web Client
- Código-fonte disponível em github.com/RaffaellaSantos/yocto_mqtt

A abordagem com Yocto Project demonstrou-se altamente eficaz para a construção de sistemas embarcados minimalistas, permitindo controle granular sobre cada pacote incluído na imagem. O uso do MQTT como protocolo de telemetria mostrou-se ideal pelo seu baixo overhead e modelo publish/subscribe que desacopla o produtor de dados do consumidor.